RIVERA TECH PROTECTION

Windows Endpoint Investigation

Suspicious Authentication & PowerShell Activity

Prepared by Marcos Rivera

Case Overview

This controlled Windows endpoint investigation was built in a Windows 11 virtual machine running in Parallels on macOS. I configured Windows security auditing, created a local FinanceUser account, simulated suspicious authentication and post-login reconnaissance, and then investigated the activity using Windows Event Viewer and PowerShell Operational logs.

The project demonstrates endpoint telemetry configuration, Windows authentication analysis, process-chain reconstruction, PowerShell investigation, evidence correlation, identification of a visibility gap, and validation of improved logging.

Environment & Tools

- Windows 11 virtual machine in Parallels
- Windows Event Viewer
- Local Security Policy and Advanced Audit Policy
- Command Prompt
- Windows PowerShell
- PowerShell Script Block Logging

Key Event IDs

- 4625 — Failed logon
- 4648 — Logon attempted using explicit credentials
- 4688 — New process created
- 4104 — PowerShell Script Block Logging

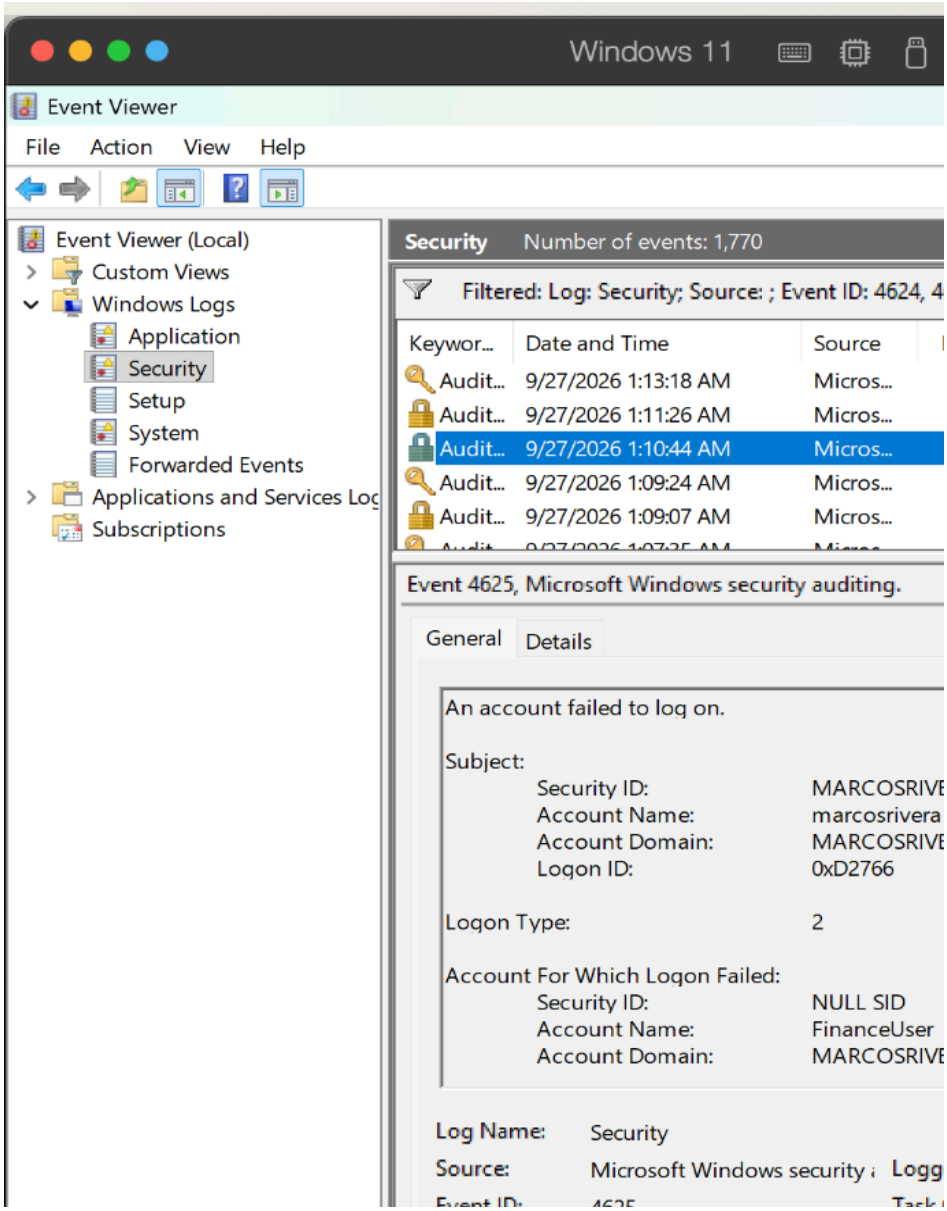
Objectives

- Generate controlled suspicious activity in a Windows endpoint lab.
- Confirm authentication activity through Windows Security logs.
- Trace process creation and parent-child relationships.

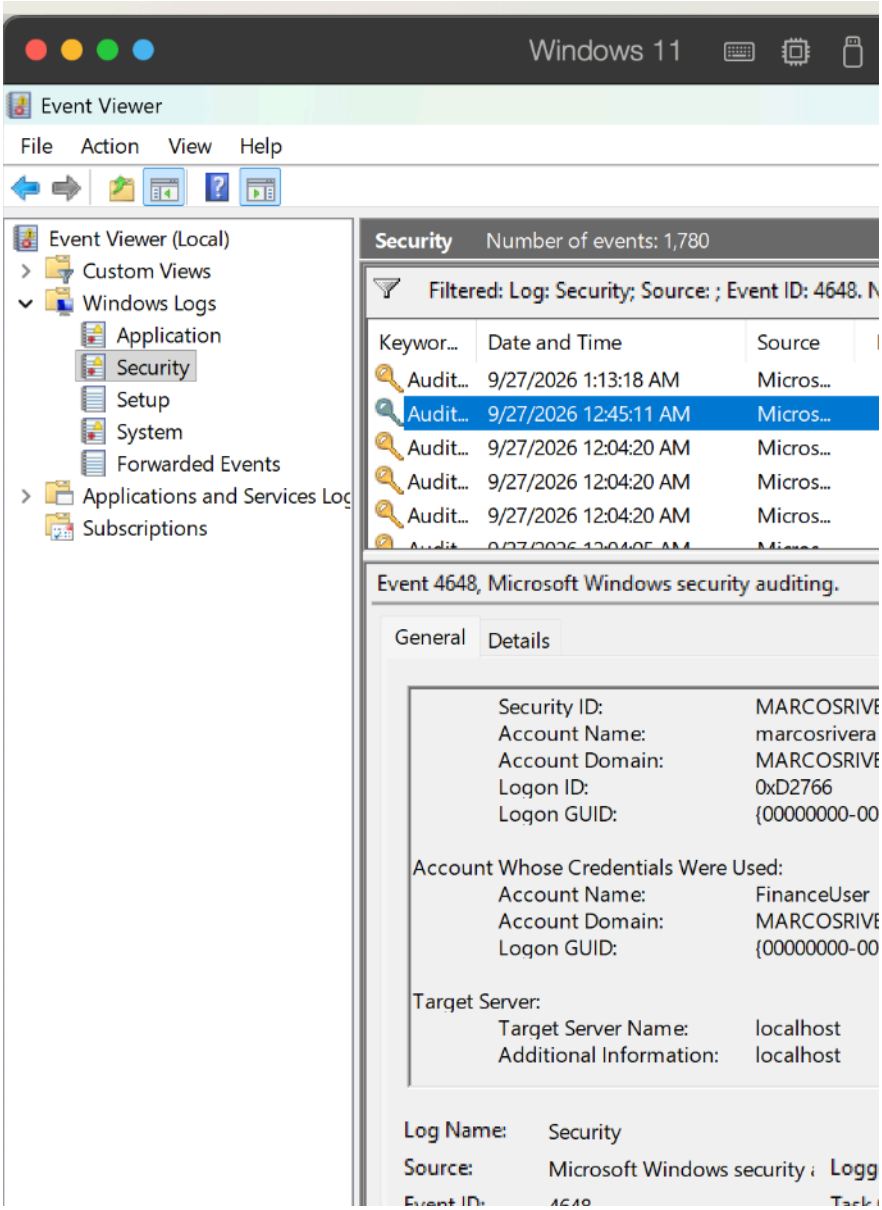
• Identify discovery commands executed under a test account.
• Investigate PowerShell execution and native cmdlets.
• Identify a telemetry limitation and improve endpoint visibility.
• Build an evidence-based incident timeline without overstating conclusions.

Phase 1 — Authentication Investigation

The investigation identified failed authentication activity against the local FinanceUser account. Event ID 4625 recorded a failed logon with Logon Type 2, indicating an interactive local logon attempt.



A separate Event ID 4648 recorded the normal marcosrivera account using explicit credentials for FinanceUser against localhost. This activity aligned with the controlled use of alternate credentials during the simulation and provided a bridge between the initiating account and the FinanceUser security context.

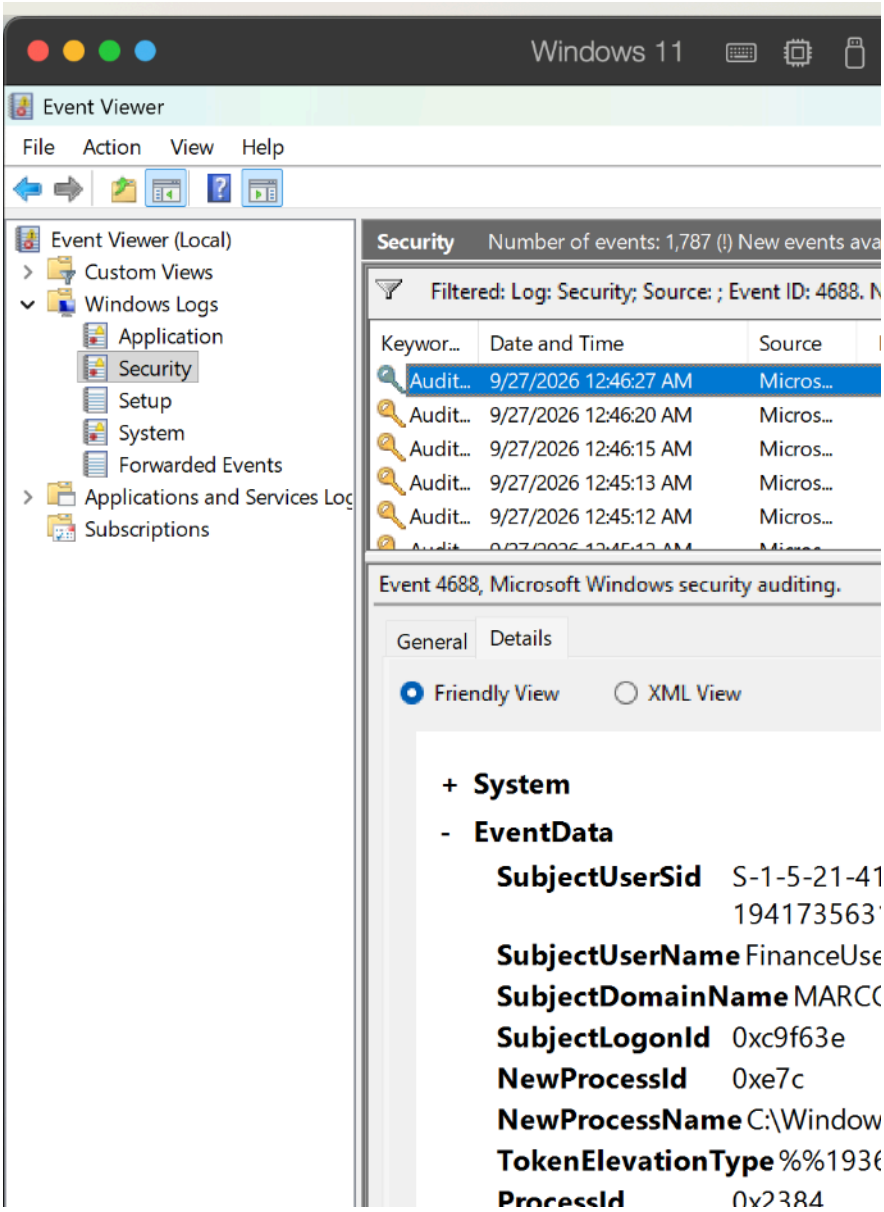


Analyst Finding

The authentication evidence established that FinanceUser was actively targeted during the lab and that explicit credentials were later used to execute activity under that account. Rather than assuming every successful logon would appear as a Type 2 event, I correlated the event type with the authentication method actually used.

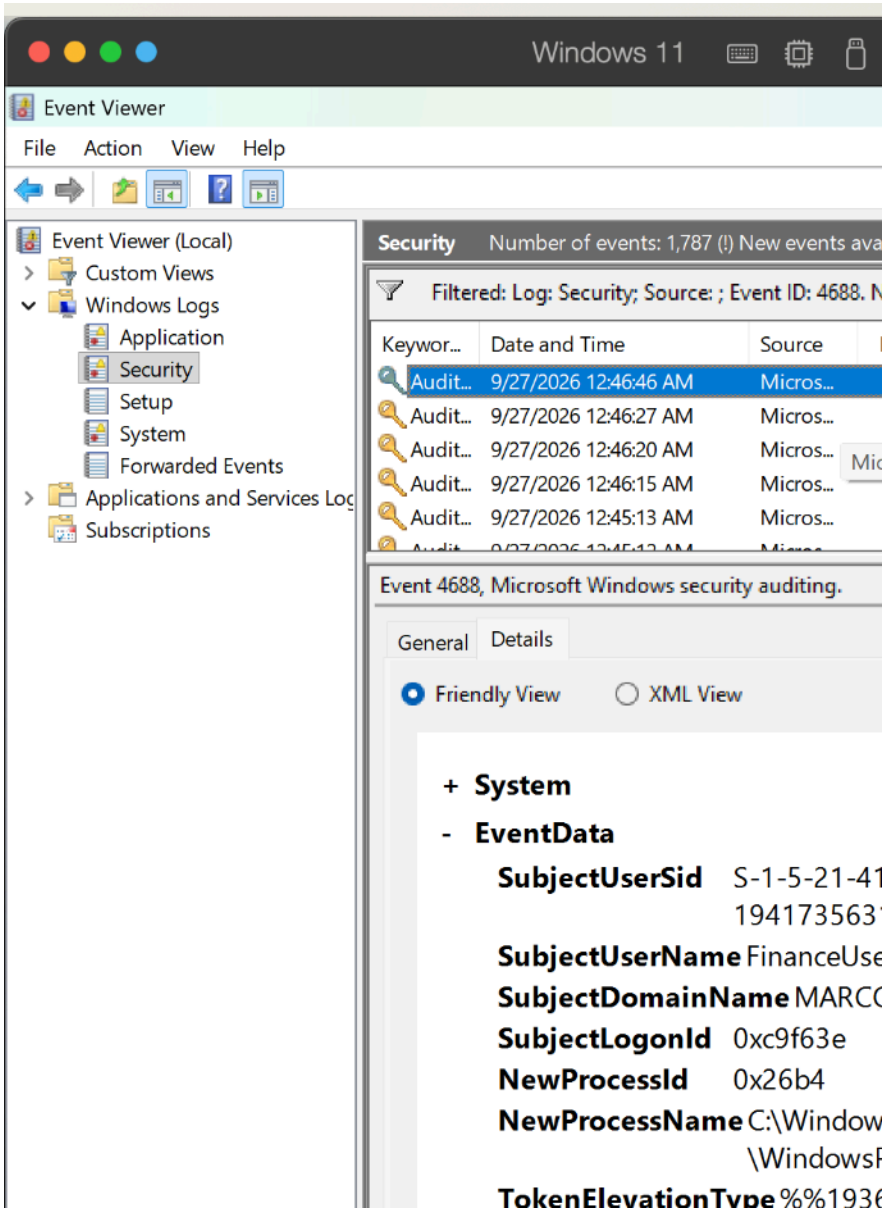
Phase 2 — Process Creation & Discovery

Event ID 4688 was used to reconstruct activity performed under FinanceUser. The process creation logs confirmed several discovery commands, including whoami.exe, hostname.exe, and ipconfig.exe.



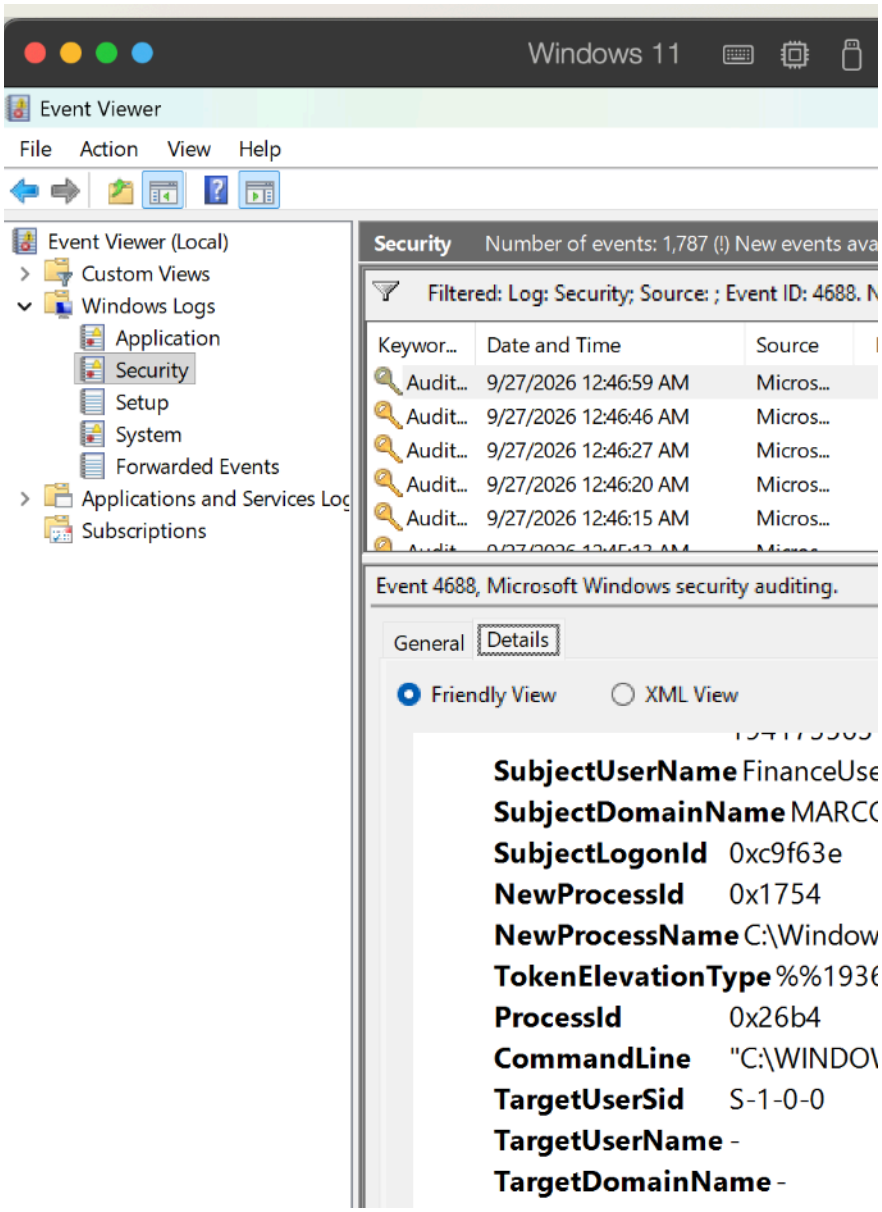
At 12:46:46 AM, FinanceUser launched powershell.exe. The creator process was C:\Windows\System32\cmd.exe, establishing the parent-child relationship:

cmd.exe → powershell.exe



Shortly afterward, Event ID 4688 recorded whoami.exe executing with powershell.exe as the parent process. This established a deeper process chain:

FinanceUser → cmd.exe → powershell.exe → whoami.exe



Analyst Finding
The commands observed are legitimate Windows utilities. However, when grouped together immediately after suspicious authentication activity, whoami, hostname, and ipconfig are consistent with post-authentication discovery and reconnaissance. The evidence supports a suspicious pattern, but the controlled nature of the lab means no malicious compromise is claimed.

Phase 3 — PowerShell Telemetry Gap

The initial Security and PowerShell Operational logs proved that PowerShell launched, but they did not expose every native PowerShell cmdlet entered during the session. In particular, Get-Process

and Get-Content executed inside the existing powershell.exe process and therefore did not create separate 4688 events.

This exposed a telemetry limitation: process creation auditing alone was not sufficient to reconstruct all PowerShell activity.

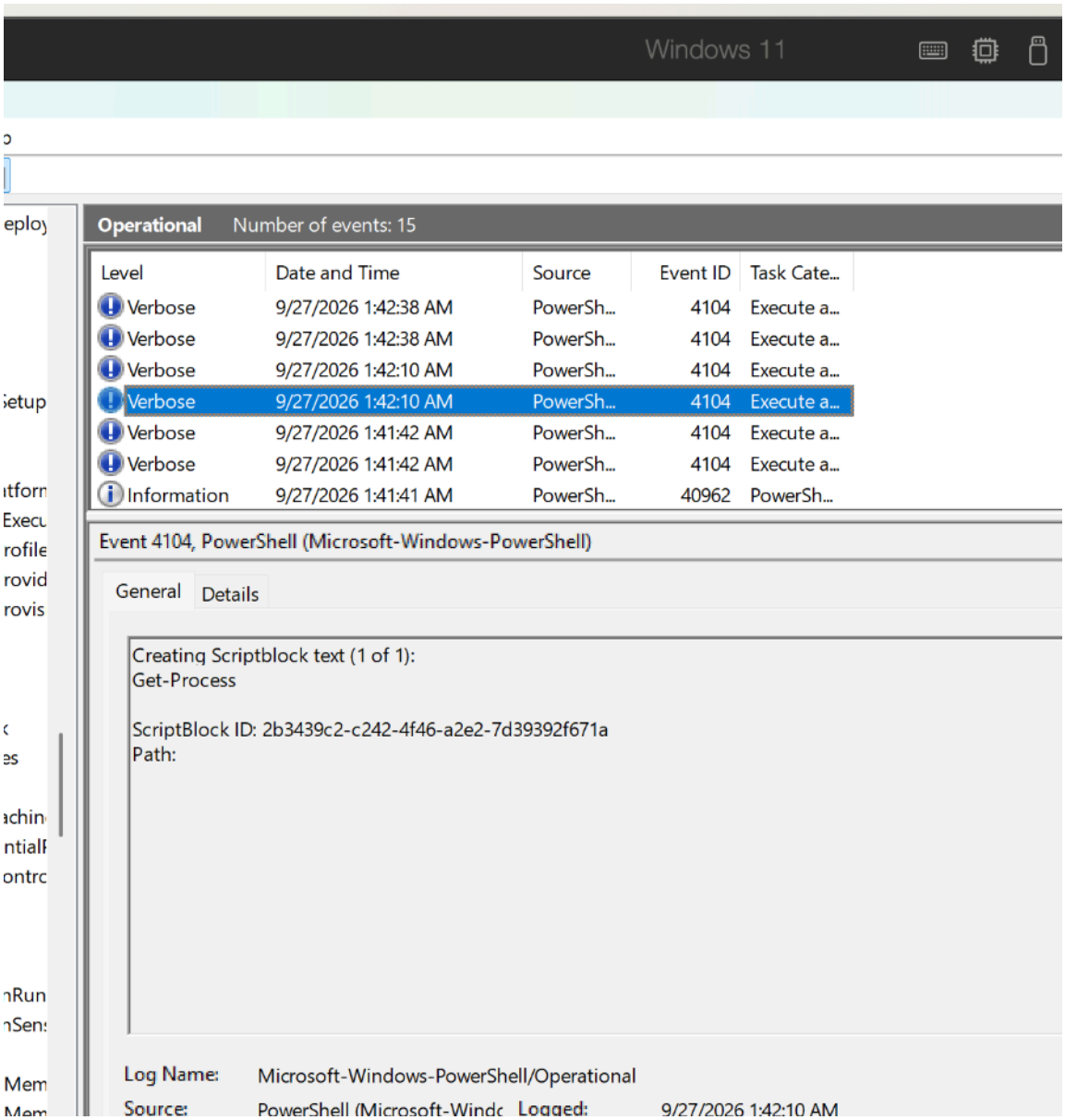
Logging Improvement

I enabled PowerShell Script Block Logging through Group Policy and repeated a small controlled portion of the simulation. The improvement was then validated through Event ID 4104.

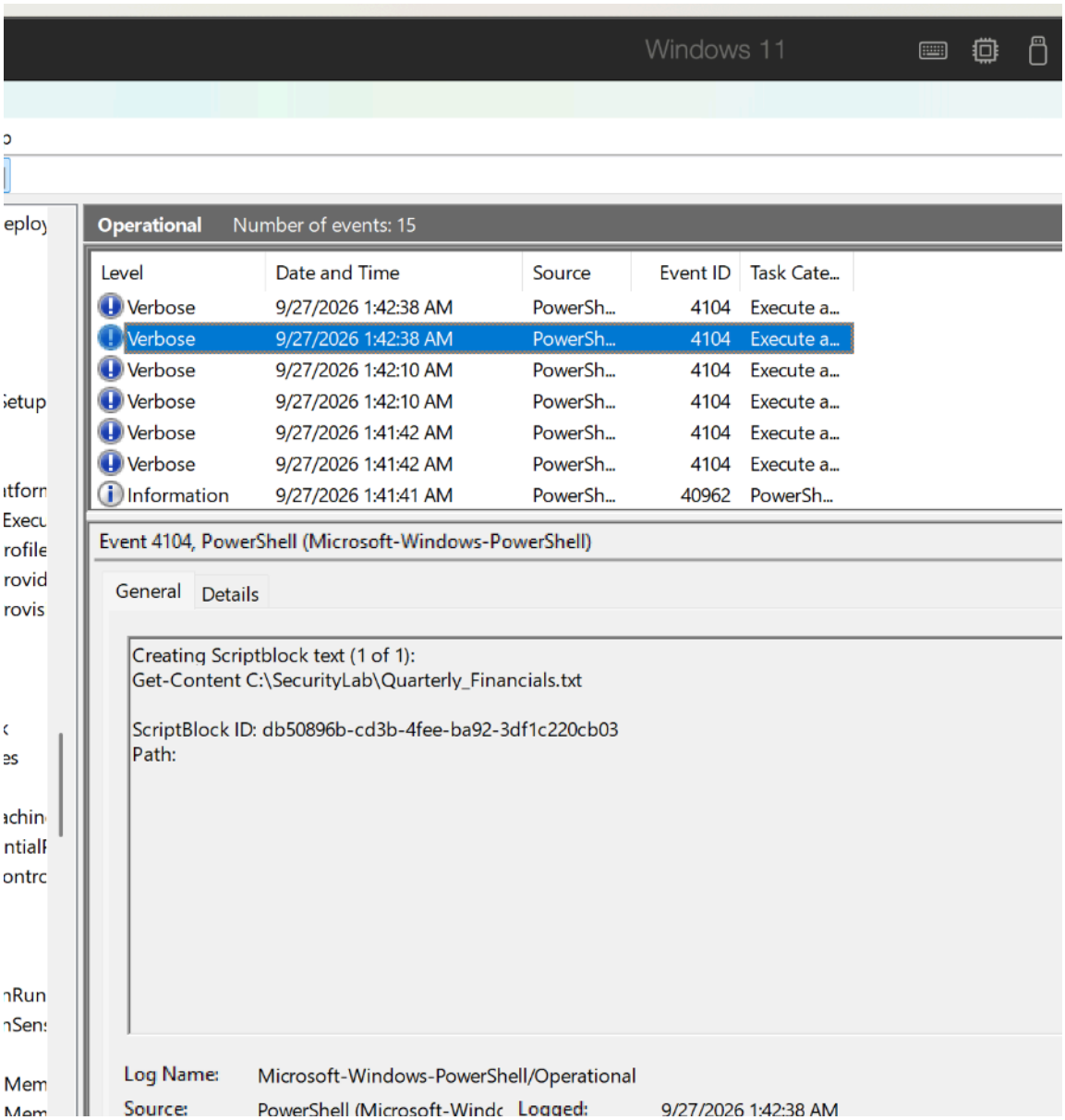
Phase 4 — Script Block Validation

After enabling Script Block Logging, Event ID 4104 successfully captured the native PowerShell commands.

Get-Process:



Get-Content C:\SecurityLab\Quarterly_Financials.txt:



This confirmed that the telemetry enhancement successfully increased visibility into commands executed within PowerShell.

Evidence Timeline

12:45:11 AM — Event 4648 recorded explicit credentials for local FinanceUser on localhost.
12:46 AM — Event 4688 recorded FinanceUser discovery-related process creation.
12:46:46 AM — powershell.exe launched from cmd.exe.
12:46:59 AM — whoami.exe executed from powershell.exe.
12:46 AM — hostname.exe and ipconfig.exe executed under FinanceUser.
1:10:44 AM — Event 4625 recorded a failed interactive logon for FinanceUser.

1:42 AM — Event 4104 captured Get-Process and Get-Content after Script Block Logging was enabled.

Analyst Conclusion

In this controlled lab, the observed activity was suspicious and consistent with post-authentication reconnaissance. By correlating Windows Security events with PowerShell Operational logs, I reconstructed authentication activity, process execution, discovery behavior, PowerShell usage, and access to a simulated sensitive financial file.

A key finding was the limitation of process creation telemetry for native PowerShell cmdlets. I addressed that gap by enabling Script Block Logging and validated the improvement through Event ID 4104. This demonstrated not only log analysis, but also the ability to recognize insufficient visibility, improve telemetry, and confirm that the new control worked as intended.

Recommendations

- Maintain process creation auditing with command-line capture on monitored Windows endpoints.
- Enable PowerShell Script Block Logging where organizational policy and privacy requirements permit.
- Correlate authentication events with process creation rather than evaluating logon events in isolation.
- Investigate clusters of discovery commands following unusual authentication activity.
- Establish alerting for suspicious PowerShell execution chains and unexpected alternate-credential use.
- Preserve endpoint logs centrally in a SIEM to support correlation and historical analysis.

Skills Demonstrated

- Windows Event Analysis
- Authentication Investigation
- Windows Security Event Correlation
- Process Creation Analysis
- Parent-Child Process Reconstruction
- PowerShell Logging
- Endpoint Telemetry Configuration
- Incident Timeline Reconstruction
- Evidence-Based SOC Analysis
- Blue Team Investigation

Portfolio Summary

This project demonstrates my ability to build and configure a Windows security lab, simulate controlled adversary-like activity, investigate the resulting telemetry, identify a logging gap, improve endpoint visibility, and communicate findings in a structured security investigation.